Índice

1 – Introducción....1
2 - Objetos de javascript....2
2.1 Date....2
2.2 Math....4
2.3 Number....5
2.4 String....7
2.5 Navigator....8
2.6 Screen....9
2.7 Window....10
2.8 Document....12
2.9 History....14
2.10 Location....15
3 Generación de elementos HTML desde código javascript....17
4 Gestión de las ventanas....17
4.1 Abrir y cerrar nuevas ventanas....18
4.2 Apariencia de las ventanas....18
4.3 Comunicación entre ventanas....19

1 – Introducción.

Propiedades, métodos y eventos son conceptos fundamentales para comprender el funcionamiento de un objeto.

- Las propiedades son las características de un objeto. Por ejemplo, una de las propiedades de una página puede ser su color de fondo (bgColor).
- Los métodos son funciones o tareas que pueden realizar los objetos. Por ejemplo, el método **sqrt()** del objeto **Math** realiza el cálculo de la raiz cuadrada de un número.
- Los eventos son situaciones que pueden llegar a realizarse o no. Por ejemplo, podemos detectar si el usuario pulsa el botón derecho del ratón. Cada objeto puede detectar una serie de eventos.

En esta unidad estudiaremos las propiedades y métodos de los objetos.

2 - Objetos de javascript

Javascript proporciona una serie de objetos. Se ordenan de modo jerárquico :

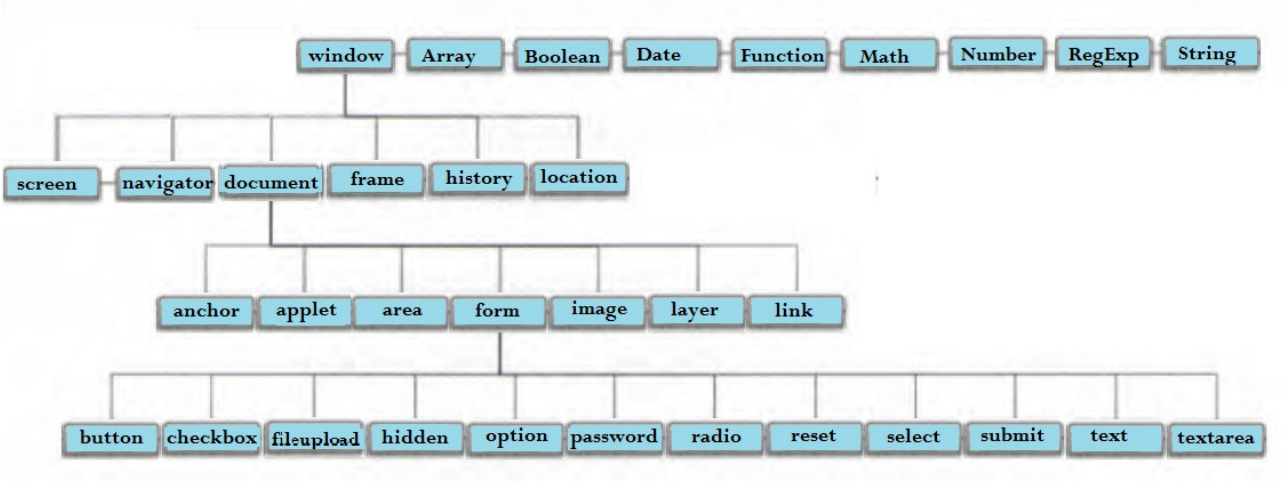

Figura *Jerarquía de los objetos de JavaScript*

La creación de un objeto se lleva a cabo a través de la palabra clave **new**. Ejemplo :

var objeto_mio = new Object(); // Creo un objeto. Puede ser predefinido por javascript o propio.

Accedemos a sus propiedades y métodos con el operador punto (.)

objeto_mio.propiedadtal;
objeto_mio.metodocual([parámetros o no]);

Veremos en la siguiente UD la creación de objetos personalizados.

2.1 Date

Sobre la clase Date recae todo el trabajo con fechas en Javascript, como obtener una fecha, el día la hora actuales y otras cosas. Para trabajar con fechas necesitamos instanciar un objeto de la clase Date y con él ya podemos realizar las operaciones que necesitemos.

Un objeto de la clase Date se puede crear de dos maneras distintas. Por un lado podemos crear el

objeto con el día y hora actuales y por otro podemos crearlo con un día y hora distintos a los actuales. Esto depende de los parámetros que pasemos al construir los objetos.

Para crear un objeto fecha con el día y hora actuales colocamos los paréntesis vacíos al llamar al constructor de la clase Date.

var miFecha = new Date()

Para crear un objeto fecha con un día y hora distintos de los actuales tenemos que indicar entre paréntesis el momento con que inicializar el objeto. Hay varias maneras de expresar un día y hora válidas, por eso podemos construir una fecha guiándonos por varios esquemas. Estos son dos de ellos, suficientes para crear todo tipo de fechas y horas.

miFecha = new Date(año,mes,dia,hora,minutos,segundos)
miFecha = new Date(año,mes,dia)

Los valores que debe recibir el constructor son siempre numéricos. Un detalle, el mes comienza por 0, es decir, enero es el mes 0. Si no indicamos la hora, el objeto fecha se crea con hora 00:00:00.

Los objetos de la clase Date no tienen propiedades pero si un montón de métodos, vamos a ver la mayoría de ellos.

getDate()	Devuelve el día del mes. Número entre 1 y 31.
getDay()	Devuelve el día de la semana. Entre 0 (domingo) y 6 (sábado)
getFullYear()	Retorna el año con 4 dígitos.
getHours()	Retorna la hora. Entre 0 y 23.
getMilliseconds()	Devuelve los milisegundos entre O y 9999.
getMinutes()	Devuelve los minutos. Entre 0 y 59.
getMonth()	Devuelve el mes. Entre 0 (enero) y 11 (diciembre)
getSeconds()	Devuelve los segundos. Entre 0 y 59.
getTime()	Devuelve los milisegundos transcurridos entre el día 1 de enero de 1970 y la fecha correspondiente al objeto al que se le pasa el mensaje.
parse()	Analiza una fecha y devuelve el número de milisegundos pasados desde el 1 de enero de 1970 hasta la fecha analizada
setDate()	Actualiza el día del mes.
setFullYear()	Cambia el año de la fecha al número que recibe por parámetro.

setHours()	Actualiza la hora.
setMilliseconds()	Establece el valor de los milisegundos.
setMinutes()	Cambia los minutos.
setMonth()	Cambia el mes (atención al mes que empieza por 0).
setSeconds()	Cambia los segundos.
setTime()	Actualiza la fecha completa. Recibe un número de milisegundos desde el 1 de enero de 1970.
toDateString()	Convierte la fecha del objeto Date en una cadena de caracteres.
toTimeString()	Convierte la parte de tiempo de un objeto Date en una cadena

EJEMPLO : Obtenemos la fecha actual y mostramos los 100 primeros días a partir de esa fecha.

```
<script type="text/javascript">
            var f = new Date();  // Obtenemos la fecha actual
            var i = 0;
            for (i = 1 ; i <= 100 ; i++ ) {
                  document.write(" Fecha : " + f.getDate() + "/" + f.getMonth()
+ "/" + f.getFullYear()
+" <br> " );
                  f.setDate(  f.getDate() + 1 ) ;
            }
</script>
// Observa el campo mes en este ejemplo. ¿ como lo solucionarías ?
```

2.2 Math

Permite realizar operaciones matemáticas. Entre sus propiedades están :

E	Devuelve la constante de Euler.
LN2	Devuelve el logaritmo natural de 2.
LN10	Devuelve el logaritmo natural de 10.
LOG2E	Devuelve el logaritmo de E en base 2.
LOG10E	Devuelve el logaritmo de E en base 10.
PI	Devuelve el valor de Pi.
SQRT1_2	Devuelve la raíz cuadrada de 1/2.

SQRT2	Devuelve la raíz cuadrada de 2

Métodos :

abs()	Devuelve el valor absoluto.
acos() asín() atan() cos() sin() tan ()	Funciones trigonométricas. Arcocoseno, arcoseno, arcotangente, coseno, seno, tangente
ceil()	Devuelve el número entero superior.
exp()	Devuelve el exponencial.
floor()	Devuelve el número entero inferior.
log()	Devuelve el logaritmo natural.
max()	Devuelve el número máximo entre los números pasados como argumento.
min()	Devuelve el número mínimo entre los números pasados como argumento.
pow()	Devuelve el resultado de un número elevado a una potencia pasada como argumento.
random()	Devuelve un número aleatorio entre O y 1. O <= x < 1
round()	Redondea un número al número entero más próximo.
sqrt ()	Devuelve la raíz cuadrada.

Para utilizar el objeto Math no es necesario definir un objeto con el operador new. Ejemplo :

```
var radio =  prompt("Teclea el radio de un círculo ");
var area = Math.PI * Math.pow( radio, 2);
alert("El area del círculo es de : " + area );
```

2.3 Number

Este objeto representa el tipo de dato número con el que trabaja Javasript. Podemos asignar a una variable un número, o podemos darle valor, mediante el constructor Number, de esta forma:

```
b = new Number(valor);
```

Por ejemplo, b = new Number(3.2); da a b el valor 3.2. Si no pasamos algún valor al constructor, la variable se inicializará con el valor 0. No es normal utilizar esta forma de crear datos numéricos.

Propiedades :

MAX_VALUE	Devuelve el mayor número posible en JavaScript.
MIN_VALUE	Devuelve el menor número posible en JavaScript.
NaN	Representa el valor especial Not a Number
NEGATIVE_INFINITY	Representa el infinito negativo
POSITIVE INFINITY	Representa el infinito positivo.

Métodos :

toExponential()	Convierte el número en una notación exponencial
toFixedO	Formatea el número con la cantidad de dígitos decimales que pasemos como parámetro. Redondea.
toPrecision()	Formatea el número con la longitud que pasemos como parámetro. Devuelve un string.

EJERCICIO :
Prueba el siguiente código :

```
<script type="text/javascript">
alert("Propiedad MAX_VALUE: " + Number.MAX_VALUE) ;
alert("Propiedad MIN_VALÜE: " + Number.MIN_VALUE) ;
alert("Propiedad NaN: " + Number.NaN) ;
alert("Propiedad NEGATIVE_INFINITY: " + Number.NEGATIVE_INFINITY) ;
alert("Propiedad POSITIVE_INFINITY: " + Number.POSITIVE_INFINITY) ;
var NI = 3.141592653589793;
alert("Pi formateado: " + NI.toPrecision(3));
</script>
```

Ejemplo de toPrecision() :

```
var num = 13.3714;
var a = num.toPrecision();
var b = num.toPrecision(2);
var c = num.toPrecision(3);
```

```
var d = num.toPrecision(10);
```

El resultado :

13.3714
13
13.4
13.37140000

2.4 String

Permite manipular las cadenas de texto.

Propiedades :

length	Corresponde a la longitud de la cadena

Algunos métodos :

charAt(num)	Permite acceder a un carácter en concreto de una cadena
indexOf(string)	Devuelve la posición de la primera ocurrencia del carácter pasado como parámetro.
lastIndexOf(string)	Devuelve la posición de la última ocurrencia del carácter pasado como parámetro
match( )	Busca una coincidencia en una cadena y devuelve todas las coincidencias encontradas.
replace(cadena_o_expr, sustituto)	Busca una coincidencia en una cadena y si existe, la remplaza por otra cadena pasada como parámetro.
search( )	Busca una coincidencia en una cadena y devuelve la posición de la coincidencia.
slice()	Extrae una parte de una cadena en base a los parámetros que indiquemos como índices de inicio y final.
split()	Corta una cadena en base a un separador que pasamos como parámetro
substr(inicio, longitud)	Devuelve una subcadena en base a un índice y longitud pasados como parámetros.

substring(inicio, fin)	Devuelve una subcadena en base a un índice de inicio y de final pasados como parámetros.
toLowerCase( )	Devuelve la cadena en minúsculas. No la cambia.
toUpperCase()	Devuelve la cadena en mayúsculas. No la cambia.

Los métodos del objeto string no respetan los estándares de la W3C *(World W/de Web Consortium),* organismo encargado de la estandarización de las tecnologías de Internet. Por este motivo, es importante prestar mucha atención al uso de estos métodos y, en muchos casos, es preferible el uso de un diseño basado en hojas de estilo en cascada (CSS). Este comentario se debe a métodos como fontsize() o fontcolor() no añadidos en la tabla anterior.

EJEMPLOS :

```
var a = "Hola jose luis";
alert ("La variable a tiene " + a.length + " caracteres");
var i = a.indexOf("jose");
alert("La palabra jose se encuentra en la posicion : "+ i );  // En este caso i
valdrá : 5
                                                    // Si no está devuelve -1
i = a.indexOf("jose", i + 1 ) ;  // Busca "jose" a partir de la posición 6
alert( a.toUpperCase() ) ; // Muestra HOLA JOSE LUIS
alert( a ) ; // Muestra Hola jose luis

alert( a.charAt(3) ) ;  // Muestra una a
// Para acceder a un string carácter a carácter
// De esta forma tenemos control total sobre la cadena
for(i = 0 ; i< a.length; i++) {
            alert( a.charAt(i) ) ;
            // La siguiente forma tambien es válida
            // alert( a.substr( i, 1 ) ;
      }
```

Vamos a ver ahora algunos objetos que nos permiten interaccionar con el navegador.

2.5 Navigator

Permite conocer datos como el navegador que se está utilizando, su versión y sistema operativo del usuario :

Propiedades :

appCodeName	Devuelve el código del nombre del navegador.

appName	Devuelve el nombre del navegador.
appVersion	Devuelve la versión del navegador.
cookieEnable	Determina si las *cook/es* están habilitadas o no.
platform	Devuelve la plataforma sobre la cual se está ejecutando el navegador.
userAgent	Devuelve una información completa sobre el agente de usuario, el cual es normalmente el navegador.

Métodos :

javaEnabled ()	Informa si el navegador está habilitado para soportar la ejecución de programas escritos en Java.

Ejemplo :

```
<script type="text/javascript">
      document.write("apCodename " + navigator.appCodeName );
      document.write("<p> appName " + navigator.appName );
      document.write("<p> Version " + navigator.appVersion );
      document.write("<p> cookie " + navigator.cookieEnabled );
      document.write("<p> plafform  " + navigator.platform );
      document.write("<p> Agente " + navigator.userAgent );
</script>
```

2.6 Screen

Corresponde a la pantalla utilizada por el usuario. Las propiedades son todas de lectura.

Propiedades :

availHeight	Corresponde a la altura disponible de la pantalla para el uso de ventanas(excluyendo barra de tareas)
availWidth	Anchura disponible de la pantalla para el uso de ventanas (excluyendo barra de tareas)
colorDepth	Profundidad del color
height	Altura total de la pantalla.
width	Anchura total de la pantalla.

2.7 Window

Es el objeto principal del cual "cuelgan" los demás como si fueran propiedades suyas. Se crea un objeto de este tipo para cada ventana que pueda ser lanzada desde una página Web. Se considera un objeto implícito, ya que no es necesario nombrarlo para acceder a los objetos que se encuentran bajo su nivel de jerarquía. Es decir, podemos escribir document.write("Hola"); en vez de window.document.write("Hola");

Propiedades :

closed	Indica si una ventana está cerrada o no. Booleano.
defaultStatus	Cadena que contiene el texto por defecto que aparece en la barra de estado (status bar) del navegador.
document	Documento actual de la ventana. Lo veremos en detalle.
frames	Es un array: cada elemento de este array (frames[0], frames[1], ...) es uno de los frames que contiene la ventana. Su orden se asigna según se definen en el documento HTML.
history	Se trata de un array que representa las URLS visitadas por la ventana
innerHeight	Corresponde a la altura utilizable de la ventana.
innerWidth	Corresponde al ancho utilizable de la ventana
length	Variable que nos indica cuántos frames tiene la ventana actual.
location	La URL del documento que se está visualizando. Podemos cambiar el valor de esta propiedad para movernos a otra página.
locationbar	Objeto barra de direcciones de la ventana.
menubar	Objeto barra de menús de la ventana.
name	Corresponde al nombre de la ventana
opener	Hace referencia a la ventana de navegador que abrió la ventana donde estamos trabajando.
outerHeight	Tamaño en pixels del espacio de toda la ventana, en vertical. Esto incluye las barras de desplazamiento, botones, etc.
outerWidth	Tamaño en pixels del espacio de toda la ventana, en horizontal. Esto incluye

	las barras de desplazamiento.
pageXoffset	Corresponde a la posición horizontal de la ventana.
pageYoffset	Corresponde a la posición vertical de la ventana.
parent	Referencia al objeto window que contiene el frameset.
personalbar	Corresponde a la barra de herramientas personal.
scrollbars	Corresponde a las barras de desplazamiento vertical y horizontal.
self	Corresponde a la ventana actual.
status	String con el mensaje que tiene la barra de estado.
toolbar	Corresponde a la barra de herramientas del navegador.
top	Corresponde a la ventana de nivel superior

Métodos :

alert ()	Cuadro de diálogo
back()	Regresa a la página anterior según el historial.
blur()	Elimina el foco del objeto window actual.
close()	Cierra una página
confirm()	Cuadro diálogo con botones aceptar y cancelar.
find()	Busca texto en una página.
focus()	Activa una ventana.
forward()	Avanza una página.
home()	Carga la página definida como por defecto en el navegador.
moveTo()	Mueve la ventana activa.
open()	Abre una nueva ventana.
print()	Imprime una página.
prompt()	Genera un cuadro de diálogo con un cuadro de texto para que el usuario introduzca valores.
resizeTo()	Modifica el tamaño de una ventana.
setInterval()	Evalúa una expresión cada cierto tiempo.
setTimeout()	Evalúa una expresión después de un tiempo especificado.

scrollBy()	Mueve el contenido de una ventana en función de una cantidad especificada en píxeles.
scrollTo()	Mueve el contenido de una ventana especificando una nueva posición de la esquina superior izquierda.
stop()	Detiene una página.

EJEMPLO :

Al cabo de 5 segundos de mostrar la página aparece un aviso.

```
<HTML>
<HEAD>
   <title>Ejemplo de setTimeout </title>
      <script type="text/javascript">
            function aviso() {
                  document.write( " Aviso " );
            }
      </script>
</HEAD>
<BODY>
Texto escrito en el body
      <script type="text/javascript">
            setTimeout( aviso , 5000);
      </script>
</BODY>
</HTML>
```

2.8 Document

Se refiere a los documentos que se cargan en la ventana del navegador. Con este objeto es posible manipular las propiedades y contenido de los elementos principales de las páginas web. Este objeto cuenta con una serie de subobjetos como son puntos de anclaje, imágenes, vínculos o formularios, que son los que le dan el verdadero potencial.

Propiedades :

alinkColor	Esta propiedad tiene almacenado el color en formato RGB de los enlaces activos.
anchors	Se trata de un array con los enlaces internos existentes en el documento (etiquetas <a name>)

applets	Es un array con los applets existentes en el documento (Etiqueta <applet>)
bgColor	Propiedad que almacena el color en formato RGB del fondo del documento.
cookie	Es una cadena con los valores de las cookies del documento actual. Se obtienen y definen las cookies asociadas con el documento.
domain	Guarda el nombre del servidor que ha servido el documento
embeds	Es un array con todos los EMBED del documento (etiqueta <embed>)
fgColor	En esta propiedad tenemos el color en formato RGB del primer plano de un documento.
forms	Se trata de un array con todos los formularios del documento. Los formularios tienen a su vez elementos (cajas de texto, botones, etc) que tienen sus propias propiedades y métodos.
images	Array con todas las imágenes del documento (etiqueta <images>)
lastModified	Es una cadena con la fecha de la última modificación del documento.
linkColor	Propiedad que almacena el color en formato RGB de los enlaces no visitados.
links	Es un array con los enlaces externos (etiqueta <a ref>)
referrer	Cadena con la URL del documento que llamó al actual, en caso de usar un enlace.
title	Cadena con el título del documento actual (etiqueta <title>)
URL	Cadena con la dirección del documento.
vlinkColor	Propiedad en la que se guarda el color en formato RGB de los enlaces visitados.

Métodos :

captureEvents()	Para capturar los eventos que ocurran en la página web. Recibe como parámetro el evento que se desea capturar.
close()	Cierra el flujo del documento.
getSelection ()	Devuelve un string que contiene el texto que se ha seleccionado.
handleEvent()	Activa el manejador del evento especificado.
home()	Carga la página de inicio.
open()	Abre el flujo del documento.

releaseEvents()	Libera los eventos que han sido interceptados.
routeEvents ()	Intercepta un evento y lo pasa a lo largo de la jerarquía del objeto que lo lanzó.
write()	Escribe datos en el documento.
writeln ()	Escribe datos además de un salto de línea en el documento.

Ampliaremos este objeto mas adelante.

```
// Mostramos el título y la URL del documento actual
alert( "titulo : " +  document.title );
alert( "url : "  + document.URL );
```

2.9 History

El objeto History almacena las referencias de todos los sitios web visitados.

Propiedades :

current	Corresponde a la cadena que contiene la URL de la entrada actual del historial.
length	Corresponde al número de páginas que han sido visitadas.
next	Corresponde a la cadena que contiene la siguiente entrada del historial.
previous	Corresponde a la cadena que contiene la anterior entrada del historial.

Métodos :

back( )	Carga la URL del documento anterior del historial.
forward()	Carga la URL del documento siguiente del historial.
go()	Carga la URL del documento especificado por el índice que pasamos como parámetro dentro del historial.

Ejemplo :

```
window.history.back();  // Equivalente a hacer clic en botón atrás del browser
window.history.forward(); // Idem anterior pero botón adelante.
window.history.go(-1);   // Equivale back en este caso. Go(1) va hacia adelante.
```

2.10 Location

Este objeto contiene la URL actual así como algunos datos de interés respecto a esta URL. Su finalidad principal es, por una parte, modificar el objeto location para cambiar a una nueva URL, y extraer los componentes de dicha URL de forma separada para poder trabajar con ellos de forma individual si es el caso. Recordemos que la sintaxis de una URL era:

protocolo://maquina_host[:puerto]/camino_al_recurso

Ejemplo :

http://www.marca.com:8080/champion

Propiedades :

hash	Corresponde a la cadena que representa el anclaje de la URL. Es decir, la parte de la URL que va después de la etiqueta #.
host	Cadena que contiene el nombre del servidor y el número del puerto, dentro de la URL.
hostname	Cadena que contiene el nombre de dominio del servidor (o la dirección IP), dentro de la URL.
href	Corresponde a la URL completa.
pathname	Cadena que contiene el camino al recurso, dentro de la URL.
port	Cadena que contiene el número de puerto del servidor, dentro de la URL.
protocol	Cadena que contiene el protocolo utilizado (incluyendo los dos puntos), dentro de la URL.
search	Cadena que contiene la información pasada en una llamada a un script, dentro de la URL.

Métodos :

assign()	Carga un nuevo documento.
reload()	Carga de nuevo el documento actual.
replace()	Sustituye la URL del documento actual por otra URL.

Ejemplos :
Con la siguiente URL en el navegador :
http://www.webestilo.com/javascript/ejem/ejem10.html

el código siguiente :

```
document.write("Location <b>href</b>: " + location.href + "<br>");
document.write("Location <b>host</b>: " + location.host + "<br>");
document.write("Location <b>hostname</b>: " + location.hostname + "<br>");
document.write("Location <b>pathname</b>: " + location.pathname + "<br>");
document.write("Location <b>port</b>: " + location.port + "<br>");
document.write("Location <b>protocol</b>: " + location.protocol + "<br>");
```

daría el siguiente resultado :

Location **href**: http://www.webestilo.com/javascript/ejem/ejem10.html
Location **host**: www.webestilo.com
Location **hostname**: www.webestilo.com
Location **pathname**: /javascript/ejem/ejem10.html
Location **port**:
Location **protocol**: http:

```
// Método assign()
location.assign("http://www.ejemplo.com");  // Equivalente a location.href =
"http://www.ejemplo.com"

// Método replace()
location.replace("http://www.ejemplo.com");
// Similar a assign(), salvo que se borra la página actual del array history del
navegador

// Método reload()
location.reload(true);
/* Recarga la página. Si el argumento es true, se carga la página desde el
servidor.
   Si es false, se carga desde la cache del navegador */
```

3 Generación de elementos HTML desde código javascript

Con JavaScript es posible manipular y acceder a los objetos que representan el contenido de una página. De este modo, en lugar de crear solamente documentos estáticos, es posible crear documentos dinámicos. Este proceso se puede realizar gracias al método **`document.write()`**.

Ejemplos :

```
<script type="text/javascript">

// Ejemplo 1. Imprime un texto en el documento actual.
var SO = navigator.platform;
document.write ("<h1> Documento abierto con: " + SO + " </h1>");

// Ejemplo 2. Crea una nueva ventana e introduce un texto
var texto = prompt("Ingresa un titulo para la nueva ventana: ");
var ventanaNueva = window.open();
ventanaNueva.document.write("<h1>" + texto + "</hl>");

// Ejemplo 3. Crea un formulario en el documento actual totalmente operativo.
document.write("<form name=\"cambiacolor\">");
document.write("<b>Selecciona un color para el fondo de página :</bxbr>") ;
document.write("<select name=\"color\">");
document.write("<option value=\"red\">Rojo</option>");
document.write("<option value=\"blue\">Azul</option>");
document.write("<option value=\"yellow\">Amarillo</option>");
document.write("<option value=\"green\">Verde</option>");
document.write("</select>");
document.write("<input type=\"button\" value=\"Modifica el color\"
onclick=\"document.bgColor =document.cambiacolor.color.value\">");
document.write("</form>");

</script>
```

NOTA IMPORTANTE : document.write no es la mejor forma de generar elementos html aunque por ahora lo podemos hacer así. Veremos el método correcto que es **document.createElement** mas tarde. Mientras tanto podemos utilizar esta opción o la siguiente :

```
var texto = "<h1> Insertando texto h1 dentro de un div </h1>";
document.getElementById("demo").innerHTML = texto;
```

Siendo el html :

```
<div id="demo"> </div>
```

4 Gestión de las ventanas

En los siguientes apartados veremos cómo abrir y cerrar nuevas ventanas, cómo controlar la apariencia de las mismas y cómo crear una comunicación e interacción entre dos o más ventanas.

4.1 Abrir y cerrar nuevas ventanas

En html se puede abrir una nueva ventana con :

<a href="mipagina.html" target="_blank">

En javascript :

```
window.open(URL nombre_ventana ,parametros_configuracion , historia )
```

El primer parámentro es la URL de la página que deseamos mostrar en la ventana secundaria.

El segundo parámetro es el nombre que le damos a la ventana. Este nombre es utilizable desde el HTML para que sirva como target de enlaces o formularios. Es decir, si colocamos un enlace y queremos que actualice la ventana secundaria ponemos el atributo TARGET del enlace igual al nombre de la ventana.

El tercer parámetro es la configuración de la ventana, para indicar el tamaño, qué barras de menús se tienen que ver y cuales no, si tiene o no barras de desplazamiento, etc.

El último parámetro es opcional e indica :
. true → URL sustituye el documento actual en la lista del historial
. false → URL crea una nueva entrada en el historial.

Con el método window.close() se cierra la ventana.

Ejemplo :

```
// Abrimos una ventana, escribimos algo en ella y la cerramos
var ventana = window.open("","","width=300, height=200");
ventana.document.write("Escribiendo algo en la nueva ventana");
alert("Abierta ventana");
ventana.close();
```

4.2 Apariencia de las ventanas

En los parámetros de configuración son una serie de valores separados por coma. Los valores soportados son :

directories=yes|no|1|0	Obsoleto. IE only
fullscreen=yes|no|1|0	Modo pantalla completa. Valor por defecto es no. IE solo.
height=pixels	Altura de la ventana. Min. valor es 100
left=pixels	Posición izquierda de la ventana. Números negativos no permitidos.
location=yes|no|1|0	Campo de direcciones si o no. Opera solo.
menubar=yes|no|1|0	Barra de menú si o no.
resizable=yes|no|1|0	Si se puede modificar tamaño de la ventana. IE solo.
scrollbars=yes|no|1|0	Barras de desplazamiento si o no. IE, Firefox & Opera solo
status=yes|no|1|0	Barra de estado.
titlebar=yes|no|1|0	Barra de título.
toolbar=yes|no|1|0	Barra de herramientas.
top=pixels	Posicion superior de la ventana.
width=pixels	Ancho de la ventana. Min. valor es 100

4.3 Comunicación entre ventanas

La comunicación podrá ser en dos sentidos :

1 - De la ventana principal a la secundaria
2 - De la ventana secundaria a la principal.

1) De la ventana principal a la secundaria.

En el momento de la creación de la ventana secundaria guardamos la referencia de esa ventana en una variable. A partir de ese momento la variable será un sinónimo del objeto window.

```
var ventana = window.open("","","width=300, height=200,  scrollbars=yes,
status=no");
ventana.document.write("Escribiendo algo en la nueva ventana");
alert("Abierta ventana");
// Nos comunicamos haciendo referencia a ventana
ventana.document.bgColor = "red" ;
ventana.document.write("<br> mas datos");
```

2) De la ventana secundaria a la principal.

La propiedad opener del objeto window hace referencia a la ventana principal. De esta forma, si en

la ventana secundaria hacemos :

```
opener.document.bgColor = "red" ;
```

Podremos modificar la ventana principal.

Información y ejemplos podrás encontrar en :

http://www.desarrolloweb.com/manuales/40/

Actualmente Mozilla y otros navegadores dan un mensaje de error :

Uncaught DOMException: Permission denied to access property "document" on cross-origin object

Cuando intentamos trabajar con dos ventanas en "local". Para evitarlo podemos subir las páginas a un servidor o bien trabajar con Internet Explorer para probar las ventanas.